

Customer Churn Prediction and Retention System

Complete workflow, implementation and interview guide

Prepared for Kavali Harshavardhan | 30 September 2026

This portfolio project predicts which subscription customers may cancel within the next 30 days, ranks an outreach queue and recommends a next action based on billing, support and usage signals. It combines Python, SQL, machine learning and an interactive Streamlit dashboard.

Delivered	Status
Data and ML pipeline	6,600 unique synthetic customers; trained model and outputs included
Dashboard and verification	Three Streamlit pages; nine automated tests passed
Detailed guide	Workflow, implementation, results, running, publishing and interview notes
GitHub and hosting	Not published. A dedicated repository URL is still required.
Power BI	Import guide and DAX provided; no native .pbix file

Data provenance: All customer records are simulated using seed 42. The relationships between input features and churn were deliberately encoded by the simulator. The reported model performance demonstrates the pipeline; it is not evidence of real company performance or retention improvement.

How to use this guide

Start with the workflow and data sections, run the app using page 10, then review the evaluation and interview sections. The ZIP contains the executable source code and supporting files. This PDF explains that implementation rather than replacing its code.

Project map and business objective

Pages	Topic
3-4	Workflow, input data and target definition
5-6	Cleaning, features, split strategy and model training
7-8	Evaluation, findings and retention decisions
9-10	Dashboard, code map and local setup
11-12	GitHub, deployment, Power BI and SQL
13-14	Monitoring, testing, limitations and interview explanation

The question the system answers

At a customer snapshot date, which active customers are most likely to cancel in the next 30 days, and which of them should a retention team review first? The output is a probability and a prioritized queue, not an automatic promise that an offer will work.

Business definitions

Churn: cancellation during the 30-day outcome window after the snapshot. **Prediction time:** the snapshot itself. **Eligible population:** active customers represented in the current input batch. **Contact decision:** a threshold chosen using validation data, followed by a capacity limit in the app.

Success criteria

Technical success means reproducible data preparation, no target leakage, useful ranking relative to a baseline, valid batch predictions and a working dashboard. Business success must be measured later through an intervention experiment, considering retention lift, outreach costs and customer experience.

Technology responsibilities

Pandas and NumPy handle data; SQLite supports business queries; scikit-learn trains and evaluates models; Matplotlib draws diagnostics; joblib stores the fitted pipeline; Streamlit provides the user interface. GitHub and Docker configuration support future publishing and deployment.

The full project workflow

Stage	Action and output
1. Define	Set a 30-day target and ensure all inputs predate the snapshot.
2. Collect	Generate a reproducible synthetic source with realistic missingness and exact duplicates.
3. Clean	Remove duplicate IDs, normalize categories and validate numeric fields.
4. Explore	Compare historical churn patterns with SQL and notebook summaries.
5. Engineer	Calculate relative usage change, handling a zero denominator as missing.
6. Split	Use January-March-May for training, July for validation, September for testing.
7. Train	Fit a baseline and three ML candidates with train-only preprocessing.
8. Select	Rank by validation average precision and set an outreach cutoff.
9. Evaluate	Report one untouched holdout evaluation, curves and confusion matrix.
10. Score	Generate November customer probabilities and retention action suggestions.
11. Present	Expose risk summaries, CSV scoring and a capacity-limited queue in Streamlit.
12. Operate	Publish after account setup; monitor drift and mature outcomes; test interventions.

Data flow: raw source to cleaned records to temporal cohorts to fitted preprocessing/model to scored current customers to the outreach queue. Historical outcomes feed evaluation. They never enter the predictive feature list.

Feedback loop: once real outreach is performed, collect treatment assignment, costs and mature outcomes. Use those outcomes to evaluate retention impact and decide whether the model or action rules need revision. That real intervention loop is not simulated as proven business impact here.

Dataset, target and time windows

Field	Meaning
customer_id / snapshot_date	Artificial unique ID and prediction date; excluded from model inputs.
tenure_months	Subscription age at snapshot.
monthly_charge	Recurring INR charge per month.
usage_previous / usage_current	Usage in days -60 to -31 and -30 to -1, respectively.
support_tickets / failed_payments	Counts in the 30 days preceding the snapshot.
contract / payment_method	monthly or annual; automatic or manual.
churn_next_30d	Binary future cancellation label; target only.
usage_change	Derived ratio: (current - previous) / previous.

The raw file has 6,620 rows: 6,600 unique customers and 20 repeated rows. There are 1,100 unique customers in each of six two-month-spaced snapshots. Each customer appears once. The source has 66 missing values in each of monthly_charge, usage_current and payment_method.

Why the time boundary matters

A cancellation reason recorded after a customer leaves would reveal the answer and artificially improve evaluation. This project restricts inputs to information available before the prediction date. The target covers the future 30 days; the two-month cohort spacing allows labels to mature before the next split.

Using real data later

The provided pipeline is built around the simulator and its fixed cohorts. To use company data, replace the generator, implement point-in-time joins and outcome maturity filtering, and revise the split logic. If customers repeat across snapshots, add grouped temporal validation. Do not simply append real records to the generated files and assume the evaluation remains valid.

Cleaning, EDA and feature preparation

Cleaning implemented in code

The pipeline removes duplicate customer IDs from the generated raw data. `prepare()` checks required columns, rejects empty inputs and missing or repeated IDs, converts numeric values, and rejects negative or infinite values. It trims/lowercases categorical values and checks their allowed categories. Invalid uploads produce a readable error in the app.

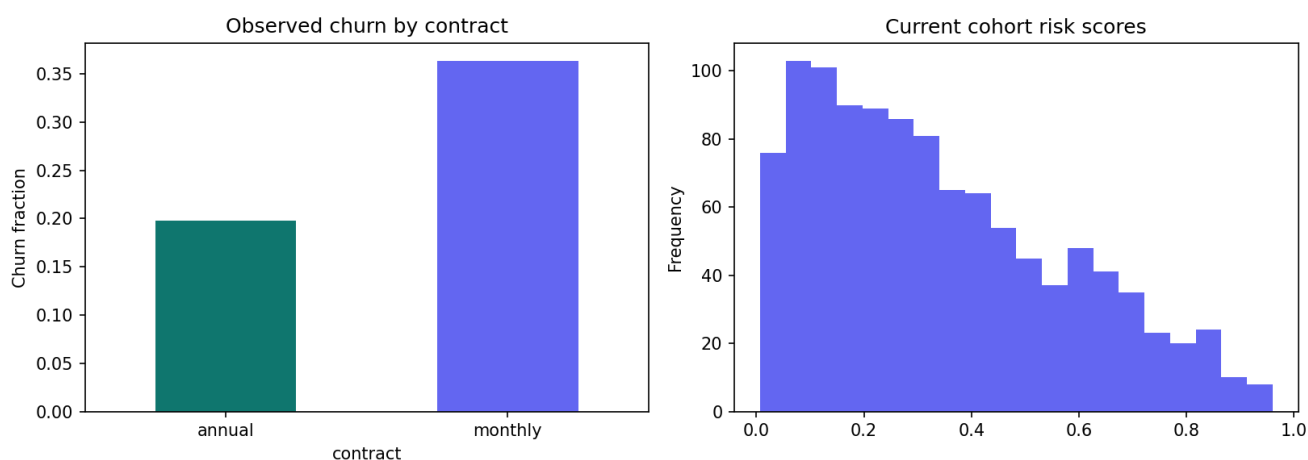
Missing values and feature calculation

```
usage_change = (usage_current - usage_previous) / usage_previous
```

When previous usage is zero, the ratio becomes missing instead of infinity. Numeric fields use the training median and categorical fields use the training mode. These transformations belong inside the fitted pipeline so validation, test and future batches reuse the training statistics. Raw missing charge values remain missing in business exports; they are not fabricated revenue.

EDA questions

What fraction of customers churn in each historical cohort? Does churn differ between monthly and annual contracts? Which fields are missing? How are current risk scores distributed? The notebook and SQL exports answer these questions, with charts in `reports/eda.png`.



The chart uses simulated observations. Associations reflect the data-generation design and are not evidence that changing a feature causes retention.

Training, selection and leakage controls

Cohort	Rows	Purpose
Jan / Mar / May 2025	3,300	Fit preprocessing and candidate models
July 2025	1,100	Choose model and contact threshold
September 2025	1,100	Untouched holdout evaluation
November 2025	1,100	Simulate current-customer scoring

Numeric fields are median-imputed and standardized. Categorical fields are mode-imputed and one-hot encoded. The ColumnTransformer and estimator are joined in a scikit-learn Pipeline. Neither `customer_id`, `snapshot_date` nor `churn_next_30d` is included in the predictive feature list.

Candidates and validation results

Model	Average precision	ROC-AUC
logistic	0.624	0.773
gradient_boosting	0.605	0.762
random_forest	0.595	0.762
baseline	0.305	0.500

The prior-probability Dummy model is the baseline. Logistic Regression estimates a linear relationship in log-odds; Random Forest averages decision trees; Histogram Gradient Boosting fits successive corrections. Logistic Regression wins by validation average precision and is stored with its preprocessing and threshold.

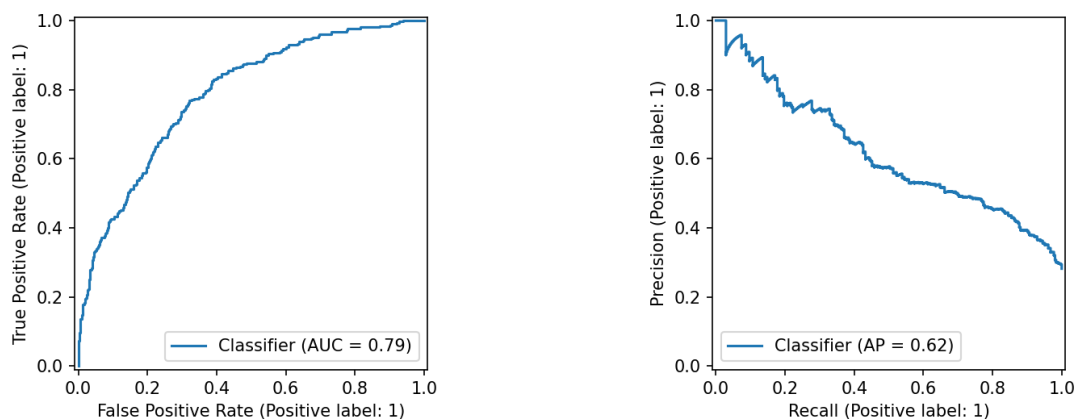
Threshold and model artifact

The strict validation cutoff is **0.5016**, chosen to flag at most 20% of the validation cohort. It is not optimized on test labels. The shipped model remains fitted on training only. `reports/model_comparison_validation.csv` reports its threshold-dependent comparison metrics at 0.5; final test metrics use the selected cutoff.

Holdout evaluation: what the numbers mean

Metric	Result	Interpretation
ROC-AUC	0.788	Ranking separation across thresholds
Average precision	0.617	Positive-class ranking; prevalence baseline is 0.282
Precision	57.6%	Fraction of flagged holdout customers who churned
Recall	49.0%	Fraction of holdout churners correctly flagged
F1	0.530	Harmonic mean of precision and recall
Brier score	0.160	Mean squared probability error; lower is better
Contact rate	24.0%	Share of holdout customers above the selected cutoff

Synthetic holdout performance



The test cohort contains 310 churners and 790 non-churners. The confusion matrix is: 678 true negatives, 112 false positives, 158 false negatives and 152 true positives. There are 264 flagged customers, of whom 152 churned.

Precision is about 2.04 times test prevalence. This is a ranking benefit within synthetic data, not a measured retention benefit. The 24% holdout contact rate also shows why a validation 20% cutoff does not guarantee the same volume on future cohorts.

From risk scores to retention decisions

The November scoring cohort contains **1,100** customers, with **270** flagged by the selected cutoff. The scored file includes probability, contact eligibility, a risk band, a suggested action and a descending-risk priority rank.

Signal (checked in order)	Suggested action
At least one failed payment	Billing assistance
Otherwise, three or more support tickets	Resolve support issues
Otherwise, usage decline greater than 25%	Offer an onboarding session
Otherwise, still eligible for outreach	Review needs with customer
Below contact threshold	Regular engagement

A flagged customer can have several problems; these rules choose the first matching action. Missing signal values do not trigger the corresponding rule. The default is a needs review. Rules are business heuristics, not individualized causal explanations from the model.

Risk bands and capacity

Low includes probabilities up to 30%; Medium covers above 30% through 60%; High covers above 60%. These display bands are independent of the contact cutoff. The app filters eligible customers and takes the top N by priority rank, where N is the chosen outreach capacity.

What drives model ranking?

Validation permutation importance measures how much average precision falls when each feature is shuffled. Usage change has the largest mean decrease (0.203), followed by support tickets (0.075) and tenure (0.068). These are global predictive associations. Correlated features can share importance, and these findings are specific to this simulator.

How to prove an action works

Randomly assign eligible customers to intervention and control. Wait for the full 30-day outcome window. Compare retention rates with uncertainty, alongside costs, margins and complaint rates. Do not report predicted revenue exposure as revenue saved.

Dashboard walkthrough and code map

Page	User interaction
Overview	Filter contract types; view current count, flagged count and mean risk; inspect risk bands; change outreach capacity; download the queue.
Score customers	Download the sample CSV, upload matching customer fields, validate the batch and download predictions. Maximum 20,000 rows.
Model evaluation	View selected model, holdout KPIs, ROC/PR curves, validation comparison and global feature importance.

Recommended demo sequence

Open Overview and change the contract filter. Explain the difference between a risk band and contact eligibility. Set an outreach limit of 100 and download the queue. Then open Score customers, download the sample and upload it again. Finally show Model evaluation and explain the synthetic-data limitation.

File / folder	Responsibility
src/core.py	Shared validation, usage feature, scoring and retention rules
src/pipeline.py	Data generation, training, evaluation, SQL database and reports
src/monitor.py	Missingness/mean-shift checks and optional labeled evaluation
app.py	Three Streamlit pages and download interactions
data/ and models/	Customer records, SQLite data, scored outputs and fitted model
reports/ and notebooks/	Metrics, charts, split IDs, findings and analysis walkthrough
tests/ and docs/	Automated checks, schema, model card and operating guides

The downloadable model file uses joblib serialization. Load only the shipped model or a model you trained yourself. The app accepts customer CSV uploads, not uploaded model files.

Run the project in VS Code

Windows setup

Install Python 3.11 or 3.12 and VS Code. Extract the project ZIP. In VS Code, choose File > Open Folder and select customer-churn-retention. Open Terminal > New Terminal. Run these commands from that folder:

```
py -3.11 -m venv .venv
.venv\Scripts\python -m pip install -r requirements.txt
.venv\Scripts\python -m streamlit run app.py
```

If Python 3.12 is installed, replace -3.11 with -3.12. Open the localhost address printed in the terminal. The fitted model and data are already included, so you do not need to train before the first app launch. Internet access is needed to install dependencies.

Reproduce and verify

```
.venv\Scripts\python -m src.pipeline
.venv\Scripts\python -m unittest discover -s tests -v
```

The pipeline regenerates the seeded source and overwrites generated data, models and reports. Preserve any custom work separately before running it. The notebook can be opened in VS Code with a Python/Jupyter kernel; its analysis cells use the generated files.

Monitoring command

```
.venv\Scripts\python -m src.monitor --input data/current_customers.csv
```

Common problems

Symptom	Fix
No module named src	Open a terminal in the project root, not inside src/.
Python version not found	Use the installed Python version in the launcher command.
Missing dependency	Install requirements using the same virtual-environment Python.
Model compatibility warning	Install the pinned package versions, or regenerate the model.
CSV rejected	Compare the sample schema; check unique IDs, categories and nonnegative numbers.

GitHub upload and deployment

Current status: the project has not been inserted into GitHub or publicly hosted. The connected account is harshakavali81-collab. Its available repositories are for other projects, and the integration cannot create a new repository. A dedicated destination is required to proceed without modifying unrelated work.

Create the destination

Open <https://github.com/new>. Name the repository customer-churn-retention, choose its visibility and create it empty, without an automatic README. Send the resulting repository URL in chat for an upload attempt, or use the commands below from the extracted project folder.

```
git init
git branch -M main
git add .
git commit -m "Build customer churn and retention system"
git remote add origin https://github.com/harshakavali81-collab/customer-churn-retention.git
git push -u origin main
```

Run each command on its own line. Change the repository owner if you use a different account.

Git may ask for authentication or a Git name/email. Use your actual identity. The included `publish_github.ps1` is an alternative if GitHub CLI is installed and signed in; it creates a private repository by default. That script has not been executed here.

Streamlit hosting

After pushing, sign into Streamlit Community Cloud with GitHub. Create an app from the repository, choose main and `app.py`, and select Python 3.11 or 3.12. Check each app page after deployment. The service installs `requirements.txt`; the committed model and generated data support startup.

Docker alternative

```
docker build -t churn-retention .
docker run --rm -p 8501:8501 churn-retention
```

Docker and hosted deployment have not been tested here. Official guide:

<https://docs.streamlit.io/deploy/streamlit-community-cloud/deploy-your-app/deploy> (checked 30 September 2026).

SQL analysis and optional Power BI

SQL deliverables

data/churn.db contains historical_customers and customer_scores. Historical data excludes the November scoring cohort. sql/analysis.sql produces four outputs: churn by snapshot, churn by contract, current risk summary and the top 100 eligible outreach customers. The generated CSV results are in reports/sql_result_*.csv.

```
SELECT contract, COUNT(*) AS customers,
       AVG(churn_next_30d) AS churn_rate
FROM historical_customers
GROUP BY contract;
```

Build the Power BI report

Import customers_clean.csv as Historical and filter snapshot_date before 2025-11-01. Import customer_scores.csv as Scores. Set IDs to text, snapshot_date to Date, charge/probability to Decimal and contact_recommended to True/False. Keep the two cohorts separate; their customer IDs do not represent a shared current/historical population.

```
Historical Customers = COUNTROWS(Historical)
Churned Customers = SUM(Historical[churn_next_30d])
Churn Rate = DIVIDE([Churned Customers],
                   [Historical Customers])
Current Customers = COUNTROWS(Scores)
Flagged Customers = CALCULATE(COUNTROWS(Scores),
                              Scores[contact_recommended] = TRUE())
Mean Predicted Risk = AVERAGE(Scores[churn_probability])
```

Create a historical page with churn rate, churn by contract and cohort trend. Create a current-risk page with flagged count, risk bands and a priority-sorted outreach table. Add contract and risk slicers. docs/POWER_BI.md contains the full measures, including known flagged monthly revenue and missing-charge count.

Revenue interpretation: the sum of known charges among flagged customers is exposure. It does not estimate actual loss, profit, treatment benefit or saved revenue. Show the count of missing charges alongside any total.

A native Power BI .pbix file is not included. Streamlit is the completed, executable dashboard. The Power BI section is a build guide using the supplied data.

Monitoring, verification and limitations

Monitoring implemented

The monitor compares a batch with training records. It flags a missing-rate change greater than 10 percentage points or an absolute numeric mean shift greater than 0.5 training standard deviations. These are review heuristics, not statistically validated drift tests. Categorical validation rejects values outside the allowed schema.

Optional labels require `customer_id` and `actual`, unique IDs, binary outcomes and coverage of the full batch. ROC-AUC additionally requires both outcome classes. The operator must ensure the 30-day window has matured; the monitor cannot establish label maturity from the supplied label schema.

Completed verification

Check	Outcome
Pipeline execution	Generated source, cleaned data, model, scores, plots and SQL outputs
Nine automated tests	Passed: app pages, schema, IDs, numeric validation, zero usage, monitoring, target exclusion, split isolation and capacity
Notebook	Analysis code cells executed against the shipped outputs
Model curve chart	Visually inspected
Not executed	Docker build, hosted CI, PowerShell upload, public hosting and native Power BI rendering

Boundaries to discuss honestly

The simulator cannot establish real-world generalization. The evaluation uses one seed and one split, without confidence intervals. Probability calibration is not explicitly fitted. No intervention lift or ROI has been measured. Real deployments require consented data, point-in-time engineering, access controls, appropriate subgroup evaluation and an experiment design.

The app is a local portfolio demonstration. It does not implement authentication or customer contact delivery. No messages have been sent to customers. The GitHub workflow is configured but can run only after the project is pushed.

Interview explanation and next actions

Your 60-second explanation

"I built a customer churn prediction and retention system using Python, SQL, scikit-learn and Streamlit. I used 6,600 synthetic customer records and defined churn as cancellation within 30 days. I cleaned duplicates and missing values, engineered usage change, and used chronological train, validation and test cohorts to avoid leakage. I compared three machine-learning models against a baseline and selected Logistic Regression using validation average precision. It achieved a holdout ROC-AUC of 0.788 on synthetic data. The app supports CSV scoring and a prioritized outreach queue. Retention recommendations are transparent rules; their business impact would need a controlled experiment."

Questions you should be ready to answer

Question	Answer
Why not accuracy alone?	A model can look accurate while missing churners. Use precision, recall and ranking metrics.
Why chronological splitting?	It better reflects predicting later customers and preserves the outcome maturity boundary.
Why a capacity-based threshold?	Outreach resources are limited. Tune using validation, then cap the current queue.
Is risk the same as treatment effect?	No. High risk does not establish that an offer will work.
What would you improve first?	Use real point-in-time data, assess calibration and subgroup performance, then test actions.

Resume bullets: use after reviewing and running the project

Built a Python and SQL churn prediction pipeline on 6,600 synthetic customer records, with chronological validation, leakage controls and three ML models compared against a baseline.

Developed a Streamlit dashboard for batch scoring, risk segmentation and capacity-limited retention outreach; documented evaluation, monitoring and limitations.

Next actions

1. Run the app and practice the demo.
2. Review reports/RESULTS.md and the code.
3. Create the dedicated GitHub repository and share its URL.
4. Publish the source and then complete hosting.
5. Add the repository and demo links to your resume only after they are live.

Authoritative project evidence: reports/metrics.json, model_comparison_validation.csv, data_quality.json and VERIFICATION.md. Full source, documentation and analysis are in the companion ZIP.